RAPPORT OCAML

Solveur Booléen

Réalisé par
Hadioui yacine et Bouamama sidahmed

Université Sorbonne Paris Nord

2024/2025

I.Présentation du problème

Ce projet a pour but d'implémenter un solveur booléen en Ocaml.
L'objectif sera de trouver toutes les solutions booléennes possibles satisfaisant un **système d'équations** où une équation est une égalité entre deux expressions booléennes.

Chacune de ces équations sont construites à partir de:

- Variables booléennes(X 1, X 2,...)
- Constantes(Vrai,Faux)
- Opérateurs logiques (Ou, Et, Non)

Ce solveur pourra trouver tous les environnements possibles(affectations possibles de valeurs Vrai ou Faux aux variables) répondant à un système d'équations et sélectionner uniquement ceux qui rendent l'équation vraie s'il en existe.

Grandes étapes du raisonnement:

→ Construire une liste de variables [X 1, X 2,...] à partir d'une équation puis utiliser celà pour créer une liste de variables à partir d'un système.

→ Construire une liste d'environnements à partir de la liste de variables, c'est-à-dire, créer une liste de liste de couples où, dans un couple, la valeur de gauche représente une variable et celle de droite une constante. On doit concevoir une liste de tous les environnements pouvant être créés à partir de la liste de variables. Si l'on possède n variables, il y aura donc 2^n environnements.
Exemple: On possède la liste [X 1, X 2] la liste d'environnement sera: [[(X 1,Vrai),(X 2,Vrai)],...,[(X 1,Faux),(X 2,Faux)]].

→ Évaluer la formule booléenne du membre de gauche de l'équation en fonction d'un environnement donné.
Vérifier si la valeur de retour de cette évaluation est identique à celle de l'évaluation du membre de droite de l'équation.
Généraliser cela pour un système qui est une liste d'équations.
Enfin on applique celà avec tous les environnements et on ne renvoie seulement ceux qui satisfont tout le système.

II.Types OCaml utilisés ;

Nous avons tout d'abord défini les types suivants pour résoudre ce problème:

```
type eb = X of int | Vrai | Faux | Et of eb * eb | Ou of eb * eb | Non
of eb;;
type valeur = Vrai | Faux;;
type env = (int * valeur) list;;
type equation = eb * eb;;
type systeme= equation list;;
```

Ici, on définit un nouveau type Ocaml eb(expression booléenne) où une eb peut être soit une variable(X 1) ou bien Vrai, Faux, ou une construction de plusieurs eb connectées via des opérateurs comme Et, Ou ou l'opérateur unaire Non.

- X 1 représente la variable logique X1
- Et(X 2 , X 3) signifie que X2 et X3 doivent être vrais

Une valeur peut prendre soit le résultat vrai soit le résultat faux, un environnement est une liste de couples composé d'un entier qui correspond à l'indice de la variable X et une valeur. Une équation est sous la forme d'un couple de deux types eb, et un système est une liste d'équations, par exemple :

- Ou (X 1, Faux) représente une équation, elle est vrai si X1 est vrai
- Faux est une valeur
- [(Ou(X 1,X 2),Vrai);(Et(X 3,Non(X 4)),X 4)] est un système d'équations
- [(1 , Vrai);(2 , Faux);(3 , Vrai)] est un environnement, cela signifie que la première et la troisième variables sont vraies et que la deuxième est fausse

III.Description des fonctions et de l'algorithme:

1. Fonctions utiles sur les listes :

Cesfonctionspermettent de manipuler leslistes :

- append : sert à concaténer deux listes

```
let rec append list1list2 = match list1 with
  | [] -> list2
  | x::ll1 -> x :: append ll1 list2;;
```

Si la 1ère est vide on renvoie la 2ème sinon on ajoute en 1er le 1er élément de la liste 1 à l'appel récursif du reste de la liste 1 et la liste 2.
Exemple: (append [1;2;3] [4;5]) renvoie [1;2;3;4;5]

- appartient : pour tester la présence d'un objet dans une liste

```
let rec appartient x list1 = matchlist1with
 | [] -> false
  | y::ll1 -> if x = y then true else appartient x ll1;;
```

Si la liste est vide on renvoie false sinon si x vaut le premier élément de la liste on renvoie vrai sinon on appelle récursivement sur x et le reste de la liste.
Exemple: (appartient 3 [1;2;3;4]) renvoie true

- union_2 : pour calculer l'union disjointe de deux liste

```
let rec union_2 list1list2 = matchlist1with
  | [] -> list2
  | x::ll1 -> if appartient x list2
      then union_2 ll1 list2
      else x :: union_2 ll1 list2;;
```

Si la 1ère est vide on renvoie la 2ème sinon, si le premier élément de la liste 1 appartient à la liste 2, on appelle récursivement sur le reste de la liste 1 est la liste 2, sinon on ajoute cet élément en 1er à l'appel récursif sur le reste de la liste 1 et la liste 2.
Exemple: (union_2 [1;2;4;5] [2;3;5;6]) renvoie [1;2;3;4;5;6]

2. Détermination des variables utilisées dans les équations:

Ilestnécessairede détecterchaquevariable présentedansunsystèmeafinde générer toutes les combinaisons possibles.

- det_var : pour déterminer une liste de variables avec une expression booléenne.

```
let rec det_var eb = matchebwith
| X b -> b::[]
| Vrai->[]
| Faux->[]
| Non x -> det_var x
| Ou(x1, x2) -> union_2 (det_var x1) (det_var x2)
 | Et(x1, x2) -> union_2 (det_var x1) (det_var x2);;
```

Évaluer tous les cas possibles d'un expression booléenne: Si c est une variable on ajoute l'entier à la liste, si c'est une constante on ne rajoute rien, si c' est Non suivi d'une expression on appelle récursivement sur l'expression et si c'est Ou ou Et, on effectue l'union disjointe des deux appels récursifs.
Exemple: (det_var Non(Et(X 2,X 1))) renvoie [2;1].

- det_var_eq : pour déterminer une liste de variables avec une équation.

```
let det_var_eq (e1,e2) =
  union_2 (det_var e1) (det_var e2);;
```

Effectuel'union disjointe des deux appels récursifs sur les membres de gauche et de droite de l'équation(du couple).
Exemple:(det_var_eq (Et(Non(X 1), X 2),Ou(Et(Non(X 3),X 1),X 4))) renvoie [2;3;1;4]

- det_list_var : pour déterminer une liste de variables avec un système.

```
let rec det_list_varsysteme = matchsystemewith
| [] -> []
  | eq::ll -> union_2 (det_var_eq eq) (det_list_var
ll);;
```

Si le système est vide, renvoie la liste vide, sinon renvoie l'union disjointe entre la fonction de détermination de liste de variables avec équation de la première équation du système et l'appel récursif sur le reste du système.

3. Fonctions de construction de la liste d'environnements.

Il faut à présent, à partir de la liste de variables déterminée juste avant Construire la liste de tous les environnements possibles.

- ajouter_devant:pour ajouter un élément au début de chaque liste dans une liste de listes.

```
let rec ajouter_devant elem liste_de_listes =
  match liste_de_listes with
  | [] -> []
  | l::ll -> (elem::l)::(ajouter_devant elem ll);;
```

Si la liste est vide on renvoie la liste est vide sinon on ajoute l'élément au début de la première liste puis on rappelle récursivement sur le reste de la liste de listes.

- temp:pour fusionner 2 listes où l'on ajoute au début 2 éléments a et b

```
let templist a b =
let avec_a = ajouter_devant a list in
let avec_b = ajouter_devant b list in
append avec_a avec_b;;
```

Exemple:
let list = [[(2,Vrai)]; [(2,Faux)]];;
let a = (1,Vrai);;
let b = (1,Faux);;
alors temp list a b vas nous donner:
[[(1,Vrai); (2,Vrai)];
[(1,Vrai); (2,Faux)];
[(1,Faux); (2,Vrai)];
[(1,Faux); (2,Faux)]]

- cons_list_env_cpl : Construit récursivement tous les environnements possibles pour une liste de variables

```
let rec cons_list_env_cpl list = match list with
  |[] -> [[]]
  |x::ll ->
      let reste = cons_list_env_cpl ll in
      let a = (x, Vrai) in
      let b = (x, Faux) in
      temp reste a b;;
```

Si la liste est vide, renvoie une liste de liste vide. Sinon, on pose a et b qui valent le couple de la première variable et Vrai/Faux. Puis, on concatène le a et le b au résultat de l'appel récursif de la fonction sur le reste de la liste.

Par exemple : pour [1;2] → [[(1,Vrai);(2,Vrai)]; [(1,Vrai);(2,Faux)]; [(1,Faux);(2,Vrai)]; [(1,Faux);(2,Faux)]]

4. Conversion de Valeurs

- var_to_bool et bool_to_var : Pour convertir une valeur en type booléen oCaml et inversement.

```
let var_to_bool var =match var with
  |Vrai->true
  |Faux->false;;
let bool_to_var bool=match bool with
  |false->Faux
  |true->Vrai;;
```

5. Évaluation d'une Expression

- eval : Évalue une expression du type eb dans un certain environnement

```
let rec eval eb env = match eb with
  | X b -> List.assoc b env
  | Vrai -> Vrai
  | Faux -> Faux
  | Et(x1, x2) -> bool_to_var (var_to_bool(eval x1 env)
&& var_to_bool(eval x2 env))
  | Ou(x1, x2) -> bool_to_var (var_to_bool(eval x1 env)
|| var_to_bool(eval x2 env))
  | Non x1 -> bool_to_var(not (var_to_bool(eval x1
env)));;
```

Si l'expression est une variable, retourne la valeur associée dans son couple présent dans l'environnement. Si c'est une valeur, elle ne change pas. Si elle utilise Et ou Ou, on effectue un et/ou logique entre la conversion booléen Caml des appels récursifs sur les expressions auquel on reconvertit le tout en valeur. De même avec le non mais on fait un not.
Exemple:
let eb = Et(X 1, Non(X 2));;
let env = [(1, Vrai);(2, Faux)];;
(eval eb env) renvoie Vrai

6. Évaluation des Équations et du Système

- satisfies_equation : Vérifie si une équation est satisfaite dans un environnement donné

```
let satisfies_equation env(x1,x2)=
 eval x1 env = eval x2 env;;
```

Renvoie true si l'évaluation du membre de gauche vaut celle du membre de droite, false sinon.
Exemple: (satisfies_equation [(1,Vrai);(2,Faux)] (Et(X 1,X 2),Non(X 1))) renvoie true car on a false des 2 côtés.

- satisfies_system : vérifie que toutes les équations d'un système sont satisfaites par un environnement donné

```
letsatisfies_systemenvsystem =
List.for_all (satisfies_equation env) system;;
```

Renvoie un et logique entre toutes les valeurs de retour de chaque élément du système c'est-à-dire, effectue un et logique pour satisfies_equation avec toutes les équations du système.
Exemple: (satisfies_system [(1,Vrai);(2,Faux)] [(Et(X 1,X 2),Non(X 1));(Ou(X 1,X 2),Vrai)]) renvoie true car l'environnement satisfie la première(true) puis la deuxième(true) et true && true=true.

- satisfies_system_allenv : évalue un système avec une liste d'environnements et renvoie seulement ceux qui la satisfont.

```
letrec satisfies_system_allenvlist_envsystem= match
list_env with
  |[]->[]
  |[[]]->[]
  |env::ll_env->if (satisfies_system env system=true)
      then env::(satisfies_system_allenv ll_env system)
      else (satisfies_system_allenv ll_env system);;
```

Si la liste d'environnements est vide, renvoie la liste vide, sinon, si le premier environnement satisfait le système, on l'ajoute à l'appel récursif sur le reste de la liste d'environnements, sinon on appelle juste récursivement sur le reste de la liste d'environnements.
Voir les exemples plus bas.

- main: fonction ultime qui fait la même chose que la précédente mais mieux niveau affichage.

```
 let mainsystem =
        satisfies_system_allenv (cons_list_env_cpl
(det_list_var( system ) ) ) system
```

IV. Exemples :

Exemple 1

Système : [(X 1, Vrai)]

- Variables : [1]
- Environnements : [[(1,Vrai)], [(1,Faux)]] ●

Environnements satisfaisants : [[(1,Vrai)]]

Exemple 2

Système : [(X 1, Non (X 2))]

- Variables : [1; 2]
- Environnements : [(1,Vrai);(2,Vrai)], [(1,Vrai);(2,Faux)], ...
- Satisfaisants : [[(1,Vrai);(2,Faux)], [(1,Faux);(2,Vrai)]]

Exemple 3

Système : [(Et (X 1, X 2), Vrai)]

- Satisfaisants : [[(1,Vrai);(2,Vrai)]]

Exemple 4

Système : [(Ou (X 1, X 2), Vrai)]

- Satisfaisants : [[(1,Vrai);(2,Vrai)], [(1,Vrai);(2,Faux)], [(1,Faux);(2,Vrai)]]

Exemple 5

Système : [(Ou (X 1, X 2), Faux)]

- Satisfaisants : [[(1,Faux);(2,Faux)]]

Exemple 6

Système : [(Non (Et (X 1, X 2)), Ou (Non (X 1), Non (X 2)))] (Lois de De Morgan)

- Satisfaisants : tous les environnements (4 au total)

Exemple 7

Système : [(Et (X 1, Non (X 1)), Faux)] (contradiction)

- Satisfaisants : tous (car c’est une tautologie que l’expression vaut toujours Faux)

Exemple 8

Système : [(X 1, X 2); (X 2, X 3)]

- Équivaut à dire : toutes les variables sont égales entre elles

- Satisfaisants : [[(1,Vrai);(2,Vrai);(3,Vrai)], [(1,Faux);(2,Faux);(3,Faux)]]

Exemple 9

Système : [(Ou (X 1, Faux), X 1)]

- Satisfaisants : tous (cette équation est toujours vraie)

Exemple 10

Système : [(Et (Ou (X 1, X 2), Non (X 3)), Vrai)]

- Satisfaisants : environnements où (X 1 || X 2) = Vrai ET X 3 = Faux

Exemple 11

Système : [(X 1,Vrai);(X 1,Faux)]

- Satisfaisants : aucun environnement satisfait ce système, renvoie []

Exemple 12

Système : [(Vrai,Faux)]

- Satisfaisants : aucun environnement satisfait ce système parce qu'il n y a pas d’environnements créés.

MERCI D’AVOIR LU LE RAPPORT.